

Contenido

1. Los módulos en node.	1
1.1. Módulos principales en Node.	1
1.2. Usando nuestros propios módulos.	2
1.3. Módulos de terceros. NPM.	4
1.4. Instala módulos locales en un proyecto	4
1.5. Instalar módulos globales en el sistema.	5
2. Eventos en Node.	6
2.1. Módulo de eventos.	6
2.2. Definir un evento.	6
2.3. Asociación de eventos a objetos.	7

1. Los módulos en node.

Node.js es un marco altamente modularizado, es decir, se subdivide en numerosos módulos, bibliotecas o paquetes (a lo largo de estas notas usaremos estos tres términos indistintamente para referirnos al mismo concepto). De esta forma, sólo añadimos a nuestros proyectos aquellos módulos que necesitemos.

El núcleo de Node.js ya incorpora algunas bibliotecas de uso común. Por ejemplo:

- http y https, para que nuestra aplicación se comporte como un servidor web, o como un servidor web seguro o encriptado, respectivamente.
- fs para acceder al sistema de archivos
- utils, con algunas funciones de utilidad, como formateo de cadenas.

Para una lista detallada de módulos, podemos acceder a <https://nodejs.org/api/>. Es una API de todos los módulos incorporados en el core de Node.js, con documentación de todos los métodos disponibles en cada uno.

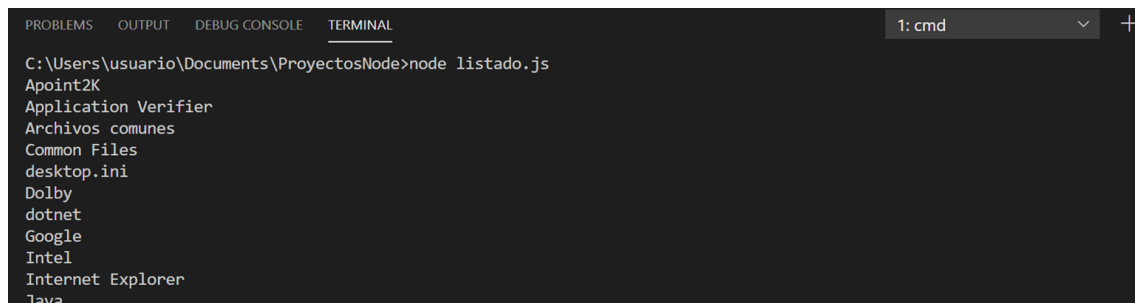
1.1. Módulos principales en Node.

Para utilizar cualquier módulo (propio de Node o de terceros) en una aplicación es necesario incluirlo en nuestro código con la instrucción requerida. Recibe como parámetro el nombre del módulo a agregar, como una cadena de texto.

Por ejemplo, vamos a crear un archivo llamado listado.js. En él vamos a hacer un pequeño programa que utiliza el módulo fs incorporado en el kernel de Node para obtener una lista de todos los archivos y subcarpetas de una carpeta específica. El código en este archivo puede verse así:

```
const route = '/Program Files'; // 'C:\\Program Files' en windows
const fs = require('fs');
fs.readdirSync(route).forEach(file => {console.log(file)});
```

Si ejecutamos este programa en la terminal (recordemos que podemos usar la terminal integrada de Visual Studio Code), obtendremos la lista de la carpeta indicada:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL 1: cmd
C:\Users\usuario\Documents\ProyectosNode>node listado.js
Apoint2K
Application Verifier
Archivos comunes
Common Files
desktop.ini
Dolby
dotnet
Google
Intel
Internet Explorer
Java
```

1.2. Usando nuestros propios módulos.

Cuando estamos haciendo un proyecto mediano o grande es conveniente descomponer nuestra aplicación en diferentes módulos. Para ello podemos crear un módulo que contendrá código independiente (instrucciones que no están dentro de una función) y funciones. El código independiente se ejecutará directamente y será necesario llamar a las funciones. Por ejemplo, dentro de la misma carpeta creamos dos archivos: utilidades.js y principal.js. El archivo de **utilidades.js** contendrá:

```
console.log('Entrando en utilidades.js');

// funciones
let Add = (num1, num2) => num1 + num2;
let Subtract = (num1, num2) => num1 - num2;

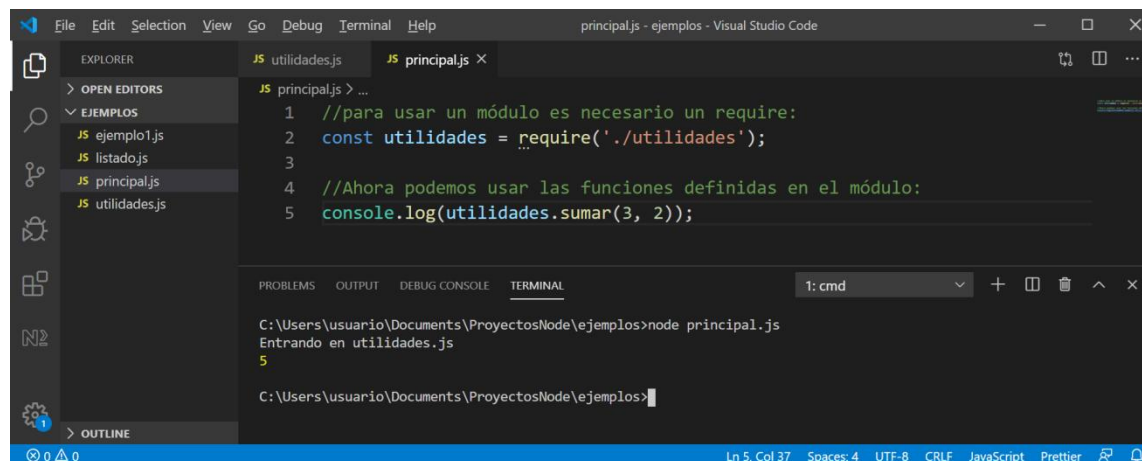
// Para exportar funciones es necesario decirlo explícitamente
module.exports = {
  Add: Add,
  Subtract: Subtract
};
```

Y el fichero **principal.js**:

```
// para usar un modulo necesitas un require:
const utilities = require('./utilidades');

// Ahora ya podemos usar las funciones definidas en el módulo:
console.log(utilities.Add(3, 2));
```

Si ejecutamos el principal:



El objeto **module.exports** admite funciones y atributos o propiedades. Por ejemplo, podríamos definir una propiedad para almacenar el valor del número "pi":

```
module.exports = {
  pi: 3.1416,
  Add: Add,
  Subtract: Subtract
};
```

El ejemplo de arriba funcionará siempre que ejecutemos la aplicación Node desde su misma carpeta:

node principal.js

Pero si estamos en otra carpeta y ejecutamos la aplicación desde allí...

C: \ OtherFolder> node C: \ ProjectsNode \ Tests \ TestsRequire \ principal.js

Entonces `require` se referirá a la carpeta desde la que estamos ejecutando, y no encontrará el archivo "utilities.js", en este caso. Para evitar este problema, podemos usar la propiedad `__dirname`, que hace referencia a la carpeta del módulo que se está ejecutando ("principal.js", en este caso):

```
const utilities = require (__dirname + '/utilidades');
```

1.3. Módulos de terceros. NPM.

En un principio, **npm** (Node Package Manager) se creó como un gestor para instalar módulos en aplicaciones de Node.js. Con el tiempo, ha evolucionado hasta convertirse en el **ecosistema de bibliotecas de código abierto más grande del mundo**, que no solo contiene módulos específicos de Node, sino también **bibliotecas de propósito general** que podemos usar en cualquier tipo de aplicación (por ejemplo, jQuery o frameworks de frontend).

Gracias a este ecosistema, podemos **reutilizar soluciones existentes** y centrarnos en las necesidades concretas de nuestra aplicación, evitando "reinventar la rueda" cada vez que necesitamos una funcionalidad común.

Todo el catálogo de librerías y módulos gestionados por npm está disponible en [npmjs.com](https://www.npmjs.com).

1.4. Instala módulos locales en un proyecto

La configuración básica de los proyectos de Node se almacena en un archivo JSON llamado "package.json". Recordamos que este archivo se puede crear directamente desde la línea de comando:

npm init

Para instalar un módulo externo en un proyecto específico, debemos abrir una terminal e ir a la carpeta del proyecto. Luego escribimos el siguiente comando:

npm install module_name

Vamos a probar un módulo sencillo y muy utilizado (tiene millones de descargas semanales), ya que contiene una serie de utilidades para facilitar el desarrollo de nuestros proyectos. Se trata del módulo "lodash", que puedes consultar en la citada web (<https://www.npmjs.com/package/lodash>). Para instalarlo escribimos lo siguiente:

npm install lodash

Después de ejecutar el comando anterior, el nuevo módulo se habrá agregado en una subcarpeta llamada "node_modules" dentro de nuestro proyecto.

Se utiliza la bandera --save para que, además de descargar el módulo, se modifique el archivo de configuración "package.json" con el nuevo módulo incluido en el bloque de dependencias:

```
"dependencies": {  
  "lodash": "^ 4.17.4"  
}
```

Para poder utilizar el nuevo módulo procederemos de la misma forma que para utilizar módulos de Node predefinidos: utilizaremos la instrucción `require` con el nombre original del módulo, por ejemplo:

```
const _ = require ('lodash');  
console.log (_ . difference ([1, 2, 3], [1]));
```

Si ejecutamos este ejemplo desde la terminal, obtendremos lo siguiente:

```
[ 2, 3 ]
```

Si decidimos subir nuestro proyecto a un repositorio en Internet como Github o similar, o dejar que alguien lo descargue para modificarlo después o simplemente copiar el proyecto en otra carpeta, **no es buena idea subir los "node_modules" carpeta**, ya que puede llegar a ser muy grande. Por lo tanto, se recomienda no compartir la carpeta "node_modules", y no hay problema en hacerlo, ya que gracias al archivo "package.json" siempre podemos ejecutar el comando:

```
npm install
```

Lo que hará que la carpeta "node_modules" se regenere.

Para desinstalar un módulo (y eliminarlo del archivo "package.json", si existe), escribimos el siguiente comando:

```
npm uninstall --save module_name
```

1.5. Instalar módulos globales en el sistema.

Para cierto tipo de **módulos**, especialmente aquellos **que se ejecutan desde una terminal** como Grunt (un administrador de tareas y automatización de Javascript) o JSHint (un verificador de sintaxis de Javascript), puede ser interesante instalarlos globalmente, para poder usarlos dentro de cualquier proyecto.

La forma de hacerlo es similar a instalar un módulo en un proyecto específico, agregando un parámetro adicional, y con la diferencia de que, en este caso, no es necesario un archivo "package.json" para administrar los módulos y las dependencias, ya que no son módulos de un proyecto, sino del sistema. La sintaxis general del comando es:

```
npm install -g module_name
```

donde el indicador -g se refiere al hecho de que desea realizar una instalación global.

Los módulos globales no se incorporan al código mediante `require`, sino que se emplean directamente como comandos en la terminal para facilitar el desarrollo, como sucede con herramientas muy comunes en Node.js tales como nodemon, eslint o typescript.

2. Eventos en Node.

Muchos objetos en Node emiten eventos: un `net.Server` emite un evento cada vez que se establece una conexión, un `fs.readStream` emite un evento cuando se abre un archivo. Todos los objetos que emiten eventos son instancias de `events.EventEmitter`. Podemos usar este módulo haciendo `require("events")`;

Podemos adjuntar funciones a los objetos para que se ejecuten cuando se genere un evento. Estas funciones se denominan **listeners**.

2.1. Módulo de eventos

Los eventos están en un módulo separado que tenemos que requerir en nuestros programas creados con Node JS. Lo hacemos con la instrucción `"require"`:

```
const events = require('events');
```

Dentro de este módulo dispones de una serie de utilidades para trabajar con eventos. Primero veamos el emisor de eventos, que se encuentra en la propiedad `EventEmitter`:

```
let EmisorEventos = events.EventEmitter;
```

2.2. Definir un evento.

En "Node" existe un bucle de eventos, de manera que cuando se declara un evento, el sistema escucha a medida que ocurre, para luego ejecutar una función. Esta función se conoce como "callback" o como "event handler" y contiene el código que queremos que se ejecute cuando se produzca el evento al que lo hemos asociado.

Primero tendremos que instanciar un objeto de la clase `EventEmitter`, que hemos guardado en la variable `EmisorEventos` antes.

```
let ee = new EmisorEventos ();
```

Luego tendremos que usar el método `on()` para definir las funciones del manejador de eventos, o su equivalente `addEventListener()`. Para emitir un evento usamos el método `emit()`.

Por ejemplo, vamos a emitir un evento llamado "data", enviando la fecha actual como información asociada:

```
ee.emit('data', Date().toString());
```

Ahora vamos a crear una función "event handler" asociada con el evento definido en "data".

```
ee.on('data', function (date) {  
    console.log (date);  
});
```

Si queremos aprovechar algunas de las características más interesantes de las aplicaciones de NodeJS, puede ser útil usar `setInterval()` para que podamos ir enviando datos de vez en cuando:

```
setInterval(function () {  
    ee.emit('data', Date().toString());  
}, 500);
```

Con esto ya habremos construido un ejemplo de NodeJS completamente funcional. El código completo sería el siguiente:

```
const events = require('events');  
let EmisorEventos = events.EventEmitter;  
let ee = new EmisorEventos();  
  
ee.on('data', function (date) { //declaramos  
    console.log(date);  
});  
  
setInterval(function () {  
    ee.emit('data', Date().toString()); //disparar el evento  
}, 500);
```

2.3. Asociación de eventos a objetos.

Creemos una clase que define a una persona y vamos a hacer que esta persona tenga emisores de eventos asociados a la herencia.

```
const events = require('events');  
class Person extends events.EventEmitter {  
    //cada objeto de tipo Person podrá emitir eventos (emit) y escuchar  
    eventos (on).  
    constructor(name) {  
        super(); //EventEmitter no requiere parámetros en su constructor  
        this.name = name;  
    }  
}  
  
let manu = new Person('manu');  
let boris = new Person('boris');  
let people = [manu, boris];  
  
people.forEach(function (littleperson) {  
    littleperson.on('talk', function (message) {  
        console.log(littleperson.name + ' ha dicho ' + message);  
    });  
});  
manu.emit('talk', 'Espero que estudiéis node');  
boris.emit('talk', 'Lo repito mucho');
```